## Variance

$Var(X) = E(X-EX)^2$

X, Y are independent

$Var(X+Y) = Var(X) + Var(Y)$

## Baye's Rule

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)}$$

## Perceptron

$\sum_{i=1}^{d} w_i x_i >$ threshold → approve

$$h(x) = sign\left(\left(\sum_{i=1}^{d} w_i x_i\right) - thrs\right)$$

$$h(x) = sign(w^T x)$$

## Perceptron Learning A.

- Pick a miscl. point $sign(w^T x_n) \neq y_n$
- Update the weight vector

$w \leftarrow w + y_n x_n$

## Types of Learning

Supervised: unknown t.f.

Unsupervised: just input

Reinforcement: input, out, grade for out

## Hoeffding's Inequality

Sample frequency: $\nu$

Bin frequency: $\mu$ s.s $\bar{N}$

$$\mathbb{P}[|\nu-\mu|>\varepsilon] \leq 2e^{-2\varepsilon^2 \bar{N}}$$

$$\varepsilon > 0 \Rightarrow \geq 1 - 2e^{-2\varepsilon^2 N}$$

$$\mathbb{P}[|E_{in}(h) - E_{out}(h)|>\varepsilon] \leq 2Me^{-2\varepsilon^2 N}$$

$$E_{in}(h) = \frac{1}{N}\sum_{n=1}^{N} [h(x_n) \neq f(x_n)]$$

$$E_{out}(h) = \mathbb{P}[h(x) \neq f(x)]$$

$$E_{out}(g) \leq E_{in}(g) + \sqrt{\frac{1}{2N}\log\frac{2|H|}{\delta}}$$

## Pocket Algorithm

Set the pocket weight vector $\hat{w}$ to $w(0)$ of PLA

for $t = 0, \ldots, T$ do

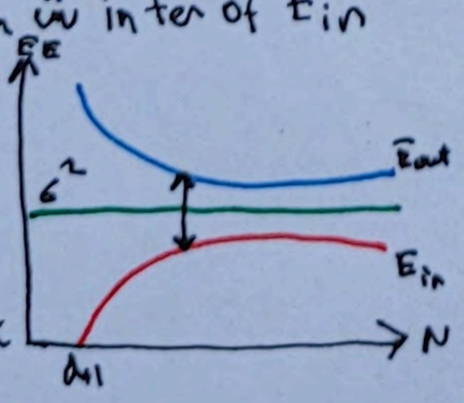
Run PLA for one update to obtain $w(t+1)$

Evaluate $E_{in}(w(t+1))$

If $w(t+1)$ is better than $\hat{w}$ in ter of $E_{in}$

set $\hat{w}$ to $w(t+1)$

return $\hat{w}$




## Linear Regression

$$h(x) = \sum_{i=0}^{d} w_i x_i = w^T x$$

$$E_{in}(h) = \frac{1}{N}\sum_{n=1}^{N}(h(x_n) - y_n)^2$$

$$w = (X^T X)^{-1} X^T y$$

## Nonlinear Transform

1) Original data $x_n \in X$

2) Transform data $z_n = \phi(x_n) \in Z$ (via $\phi$)

3) Separate data in Z-space $\tilde{g}(z) = sign(\tilde{w}^T z)$

4) Classify in X-space $g(x) = \tilde{g}(\Phi(x)) = sign(\tilde{w}^T \Phi(x))$ (via $\Phi^{-1}$)

$w^T x$ is linear in w

Any $x \xrightarrow{\Phi} z$ preserves the lin.

Ex $(x_1, x_2) \xrightarrow{\Phi} (x_1^2, x_2^2)$

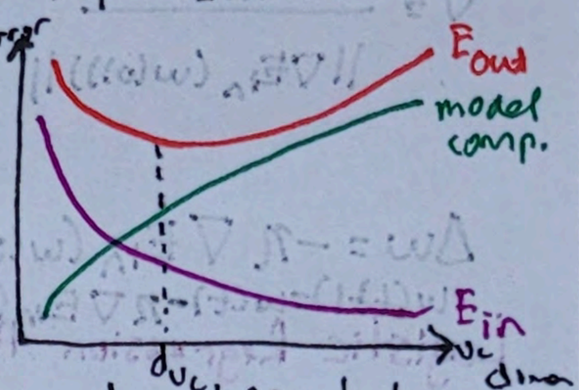
## Error Measures

Squared error: $e(h(x), f(x)) = (h(x) - f(x))^2$

Binary error: $e(h(x), f(x)) = [\![ h(x) \neq f(x) ]\!]$

$$E_{in}(h) = \frac{1}{N}\sum_{n=1}^{N} e(h(x_n), f(x_n))$$

$$E_{out}(h) = \mathbb{E}_x[e(h(x), f(x))]$$




## Noisy Targets

Target fun. isn't always func. Two identical cus → two diff. behav.

Use target distribution

## The Growth Function

$m_{\mathcal{H}}(N) \leq 2^N$ $\quad m_{\mathcal{H}}(3) = 8$ $\quad m_{\mathcal{H}}(4) = 14$

$m_{\mathcal{H}}(N) = N+1$ $\quad m_{\mathcal{H}}(N) = \frac{1}{2}N^2 + \frac{1}{2}N + 1$ $\quad m_{\mathcal{H}}(N) = 2^N$

Break point is the point of N which you fail to get all dichotomies. N=4 percep.

Positive rays $m=2$ $\quad 3 < 2^{2.3}$ $\quad m_{\mathcal{H}}(k) < 2^k$

$$B(N,k) \leq \sum_{i=0}^{k-1}\binom{N}{i}$$

$B(N,k)$: max # of dic. on N points with break point k

## The V-C Inequality

$N \leq d_{vc}(\mathcal{H})$ → $\mathcal{H}$ can shatter N pts.

$k > d_{vc}(\mathcal{H})$ k is a break for $\mathcal{H}$

$$\mathbb{P}[|E_{in}(g) - E_{out}(g)|>\varepsilon] \leq 4\, m_{\mathcal{H}}(2N)\, e^{-\frac{1}{8}\varepsilon^2 N}$$

$d_{vc} = 1$ → $\quad d_{vc} = 3$ ⟷ $\quad d_{vc} = \infty$ ⬠ $\quad d_{vc} = d+1$

**Degrees of Freedom** # of parems analog deg. $d_{vc}$: equivalent binary deg.

$$\varepsilon = \sqrt{\frac{8}{N}\ln\frac{4\, m_{\mathcal{H}}(2N)}{\delta}}$$ ⇒ we have confidence

$$E_{out} \leq E_{in} + \Omega$$

**Bias-Variance** var: how far your hyp. from the best (avg) hyp. due to finite data

bias: how far is your best hypothesis from the target func.

$$\mathbb{E}_D[[g^{(D)}(x) - f(x)]^2] = \overbrace{\mathbb{E}_D[(g^{(D)}(x) - \bar{g}(x))^2]}^{var(x)} + \overbrace{(\bar{g}(x) - f(x))^2}^{bias(x)}$$

$$\mathbb{E}_D[E_{out}(g^{(D)})] = bias + var$$

sigma: squared variance

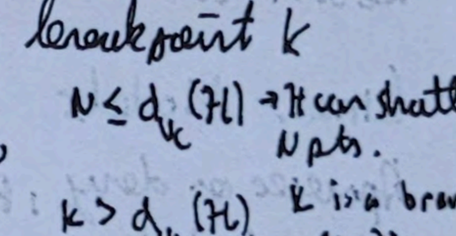
d: dimensionality of data

**Learning Curves for Linear Regression** $\sigma^2$: best approx error

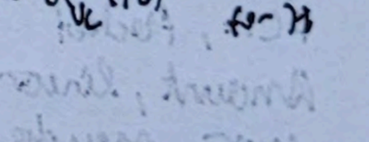
Expected in-sample $\sigma^2\left(1 - \frac{d+1}{N}\right)$, out sample $\sigma^2\left(1 + \frac{d+1}{N}\right)$

generalization $2\sigma^2\left(\frac{d+1}{N}\right)$

linear class. $s = \sum_{i=0}^{d} w_i x_i$ $\quad h(x) = sign(s)$

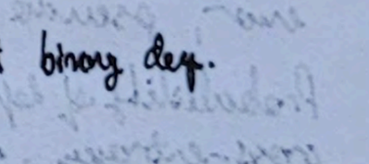



linear regression $h(x) = s$




$\theta(s) = \frac{e^s}{1+e^s}$

logistic regression $h(x) = \theta(s)$




logistic regression $E_{in}(w) = \frac{1}{N}\sum_{n=1}^{N} \ln(1 + e^{-y_n w^T x_n})$

$$P(y|x) = \begin{cases} f(x) & \text{for } y = +1 \\ 1 - f(x) & \text{for } y = -1 \end{cases}$$

Data $(x, y)$ with binary y generated by noisy target

$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

## Likelihood

$$\prod_{n=1}^{N} P(y_n \mid x_n) = \prod_{n=1}^{N} \theta(y_n w^T x_n)$$

$$E_{in}(w) = \frac{1}{N}\sum_{n=1}^{N} \overbrace{\ln(1+e^{-y_n w^T x_n})}^{\text{cross entropy error}}$$

## Gradient Descent

General method for nonlinear opt.
Start at $w(0)$ take a step along steepest slope
Fixed step size $w(1) = w(0) + \eta \hat{v}$

$$\hat{v} = \frac{\nabla E_{in}(w(0))}{\|\nabla E_{in}(w(0))\|}$$

$\eta$ should increase with the slope
large → small

$$\Delta w = -\eta \nabla E_{in}(w(0))$$
$$w(t+1) = w(t) - \eta \nabla E_{in}(w(t))$$

## Logistic Regression Algorithm

Initialize the weights at $t=0$ to $w(0)$
for $t = 0, 1, 2 \ldots$ do
  Compute gradient

$$\nabla E_{in} = \frac{-1}{N}\sum_{n=1}^{N} \frac{y_n x_n}{1+e^{y_n w^T(t) x_n}}$$

Update the weights $w$

$$w(t+1) = w(t) - \eta \nabla E_{in}$$

Return $w$

## Linear Models

Adaline or delay, perceptron PLA, Pocket
Bound, linear reg., squared error, pseudo inverse
Probability of default, logistic regres. cross-entropy, gradient-descent

## Benefits of SGD

cheaper comp., randomi, simple

## Neural Networks

$$w_{ij}^{(\ell)} \begin{cases} 1 \le \ell \le L & \text{layers} \\ 0 \le i \le d^{(\ell-1)} & \text{inputs} \\ 1 \le j \le d^{(\ell)} & \text{outputs} \end{cases}$$

$$x_j^{(\ell)} = \theta(s_{(i,d)}^{(\ell)}) = \theta\left(\sum_{i=0}^{d^{(\ell-1)}} w_{ij}^{(\ell)} x_i^{(\ell-1)}\right)$$

$$\theta(s) = \tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}$$

## SGD

$$E_{in}(w) = \frac{1}{N}\sum_{n=1}^{N} e(h(x_n), y_n)$$

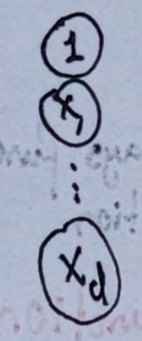
$$\Delta w = -\eta \nabla E_n(w)$$

## Backpropagation

Initialize all weights $w_{ij}^{(\ell)}$ at random
for $t = 0, 1, 2 \ldots$ do
  Pick $n \in \{1, 2, \ldots N\}$
  Forward: Compute all $x_j^{(\ell)}$
  Backward: Compute all $\delta_j^{(\ell)}$
  Update the weights $w_{ij}^{(\ell)} \leftarrow w_{ij}^{(\ell)} - \eta x_i^{(\ell-1)} \delta_j^{(\ell)}$
Return the final weights $w_{ij}^{(\ell)}$

## SGD

Initialize weights
for $t = \ldots$ do
  Pick random $x_n, y_n$
  Apply GD
  Update weights




$$\Delta e(w) = \frac{\partial e(w)}{\partial w_{ij}^{(\ell)}}$$

$$\frac{\partial e(w)}{\partial w_{ij}(\ell)} = \underbrace{\frac{\partial e(w)}{\partial s_j^{(\ell)}}}_{\delta_j^{(\ell)}} \cdot \underbrace{\frac{\partial s_j^{(\ell)}}{\partial w_{ij}^{(\ell)}}}_{x_i^{(\ell-1)}}$$

$$\Delta w_{ij}^{(\ell)} = -\eta \frac{\partial e(w)}{\partial w_{ij}^{(\ell)}} = -\eta\, \delta_j^{(\ell)}\, x_i^{(\ell-1)}$$

↳ computed by using backprop

$$\delta_i^{(\ell-1)} = \left(1 - (x_i^{(\ell-1)})^2\right) \sum_{j=1}^{d^{(\ell)}} w_{ij}^{(\ell)} \delta_j^{(\ell)}$$

Gradient Descent

$E(u,v) = (ue^v - 2ve^{-u})^2$ Start at $(1,1) = (u,v)$ in the $uv$ space $\eta = 0.1$

minimize error by gradient descent in 4 coordinate directions

a) $\frac{\partial E}{\partial u}$  b) $\frac{\partial E}{\partial v}$  c) Gradient vector $(1,1) = (u,v)$  d) $(u,v)$ update after one-step GD  e) $(u,v)$ coor. after 2 steps of GD

Explain learning curve, overfitting, how can we understand overfitting

(ii) Consider perceptron in 2D $h(x) = \text{sign}(w^T x)$ $w = [w_0, w_1, w_2]^T$ $x = [1, x_1, x_2]^T$. Technically, $x$ has 3 coordinates but we say 2D bec. first c. fixed at 1.

a) Show that the regions on the plane where $h(x) = +1$ and $h(x) = -1$ are seperated by a line. If we express this line by eq $x_2 = ax_1 + b$ what are the slope $a$ and intercept $b$ in terms of $[w_0, w_1, w_2]$

b) Draw a pic. for cases $w = [1,2,3]^T$ and $w = -[1,2,3]^T$

In more than 2D the +1 and −1 regions are seperated by hyperplane (the generalization of a line)